

EIF209 – Aplicación de Plantas: Chat en tiempo real y últimos avances

Universidad Nacional — Sede Regional Brunca Curso: EIF209 Desarrollo y Diseño de Plataformas Móviles **Actividad:** Integración de chat en tiempo real sobre la app de plantas (continuación del Laboratorio 4 — app al 90 %) **Profesor:** Daniel Granados Murillo **Estudiante:** Esteban Javier Granados Sibaja **Fecha de entrega:** 9 de junio de 2026

Última entrega: Integración de Chat en tiempo real

Esta entrega incorpora a **MyPlantCenter** un **sistema de chat en tiempo real** como nueva sección de la aplicación, consumiendo un backend de chat independiente (FastAPI + WebSocket). Cumple el enunciado:

Cada estudiante deberá agregar este chat a su aplicación de plantas en una nueva sección, en donde se vean las observaciones que se han hecho en el documento (chat grupal), además de la pantalla para los mensajes directos.

Resumen de lo entregado

- **Chat grupal** ("Observaciones del documento") — sala común en tiempo real.
- **Mensajes directos (DM)** entre usuarios conectados.
- **Indicadores de "escribiendo..."** (typing) en grupo y DM.
- **Confirmaciones de lectura** (read receipts) y **mensajes con expiración** (TTL).
- **Lista de usuarios en línea** con estado de conexión.
- **Reconexión automática** del WebSocket y *keep-alive* (ping cada 25 s).

- **Homogeneización de UI** (nuevo header centralizado reutilizable) y **corrección de bugs críticos** detectados durante la integración.

Arquitectura de la integración

El chat usa un **backend propio e independiente** del backend de plantas, con su **propio sistema de autenticación** (JWT distinto al de Firebase):

Aspecto	Backend de plantas	Backend de chat
URL	<code>https://myplantcenter.onrender.com</code>	<code>https://chat-backend-4nzg.onrender.com</code>
Autenticación	JWT de Firebase	JWT propio (<code>POST /api/chat/join</code> con <code>{nickname}</code>)
Transporte	HTTP REST	HTTP REST + WebSocket (<code>wss://.../ws/{token}</code>)

Por eso el chat **no** reutiliza el `httpClient` existente (que inyecta tokens de Firebase): se creó un cliente `fetch` independiente y un servicio de WebSocket dedicado, con el token de chat mantenido en memoria.

Estructura de la *feature* (espejo del patrón `features/`):

```
features/chat/
├─ config/chat.config.ts      - URLs (variables de entorno + fallback)
├─ types/chat.types.ts       - tipos y unión discriminada de eventos WS
├─ services/chatApi.service.ts - cliente HTTP (join, historial, usuarios)
├─ services/chatSocket.service.ts - WebSocket (reconexión 4 s, ping 25 s)
├─ context/ChatProvider.tsx   - estado global + ciclo de vida del socket
├─ hooks/useChat.ts
├─ components/ MessageBubble · MessageInput · UserListItem
└─ screens/    DMListScreen (hub) · GroupChatScreen · DMThreadScreen
```

Navegación y pantallas

Se agregó una pestaña **“Chat”** que abre un **hub** (lista) con el tab bar visible:

- Tarjeta destacada “**Chat grupal — Observaciones del documento**” en la parte superior.
- Lista de **mensajes directos** debajo.

Las conversaciones (grupal y DM) son **pantalla completa**: se oculta el tab bar flotante para que el campo de escritura no quede tapado, y usan el nuevo **header centrado con botón de regreso**. La cámara del tab bar se reubicó al **centro real** (índice 3 de 7) y se corrigió el truncado de las etiquetas.

Correcciones críticas durante la integración

Problema	Causa raíz	Solución
Los campos de texto “saltaban” en cascada (Android)	<code>returnKeyType="next"</code> activa <code>IME_ACTION_NEXT</code> ; Android traversa el foco de forma nativa y el teclado Samsung lo dispara espuriamente	Mapear a <code>IME_ACTION_DONE</code> + <code>submitBehavior="submit"</code> y desactivar el autofill
El teclado tapaba el input del chat	<code>edgeToEdgeEnabled</code> rompe el <code>adjustResize</code> clásico de Android	<code>KeyboardAvoidingView</code> con <code>behavior="padding"</code>
Los mensajes directos se duplicaban al enviarse	El mensaje “optimista” (id temporal) no se reconciliaba con el eco del servidor (id real)	Reconciliar el placeholder con el eco en lugar de añadir un segundo mensaje

Mejoras de UX y homogeneidad de diseño

- **Nuevo componente reutilizable** `DetailHeader` (título centrado + botón de regreso) como **estándar de la app** para pantallas de detalle.
- **Pantalla “Agregar planta” rediseñada**: header centralizado y formulario agrupado en secciones (“Información” y “Cuidado y detalles”) en lugar de una sola tarjeta plana.

Archivos de la integración del chat

Nuevos:

Archivo	Descripción
<code>features/chat/config/chat.config.ts</code>	URLs del backend de chat
<code>features/chat/types/chat.types.ts</code>	Tipos + eventos de WebSocket
<code>features/chat/services/chatApi.service.ts</code>	Cliente HTTP del chat
<code>features/chat/services/chatSocket.service.ts</code>	WebSocket con reconexión
<code>features/chat/context/ChatProvider.tsx</code>	Estado global del chat
<code>features/chat/hooks/useChat.ts</code>	Hook de acceso al contexto
<code>features/chat/components/MessageBubble.tsx</code>	Burbuja de mensaje
<code>features/chat/components/MessageInput.tsx</code>	Campo de escritura
<code>features/chat/components/UserListItem.tsx</code>	Ítem de usuario en lista
<code>features/chat/screens/ChatNavigator.ts</code>	Tipos del stack de chat
<code>features/chat/screens/GroupChat/GroupChatScreen.tsx</code>	Chat grupal
<code>features/chat/screens/DirectMessages/DMListScreen.tsx</code>	Hub de chats
<code>features/chat/screens/DirectMessages/DMThreadScreen.tsx</code>	Conversación directa
<code>ui/primitives/DetailHeader/DetailHeader.tsx</code>	Header reutilizable centrado

Modificados:

Archivo	Cambio
<code>core/navigation/AppNavigator.tsx</code>	Pestaña "Chat", <code>ChatProvider</code> , ocultar tab bar en conversaciones, cámara centrada
<code>core/navigation/AppTabBar.styles.ts</code>	Ajuste de etiquetas y espaciado del tab bar
<code>ui/primitives/TextField/TextField.tsx</code>	Corrección del salto de foco (IME de Android)
<code>features/auth/screens/Login/LoginV2.tsx</code> · <code>Register/RegisterV2.tsx</code>	Limpieza de <i>refs</i> de foco

Archivo	Cambio
features/plants/screens/AddPlant/AddPlantV2.tsx	Rediseño con header centralizado y secciones

Parte 1: Investigación y Análisis

1.1 Identificación de módulos faltantes

La siguiente tabla resume el estado de cada módulo antes de esta entrega (punto de partida) y después de implementar los cambios del Lab 4:

Módulo	% Antes	% Después	Estado
Autenticación	100 %	100 %	Completo
Dashboard	95 %	100 %	Completo
Gestión de Plantas	95 %	95 %	Casi completo
Escáner IA con Cámara	85 %	90 %	Casi completo
Calendario de Riego	90 %	90 %	Casi completo
Sistema de Amigos	80 %	80 %	En progreso
Explorar	75 %	80 %	En progreso
Perfil de Usuario	90 %	90 %	Casi completo
Notificaciones	0 %	90 %	Nuevo — implementado

Módulo	% Antes	% Después	Estado
Modo Vacaciones	0 %	100 %	Nuevo — funcionalidad propia
Chat (grupal + directos)	0 %	100 %	Nuevo — tiempo real (WebSocket), entrega actual
Sistema de Diseño + Navegación	100 %	100 %	Completo

Promedio de completitud (antes): ~80 % Promedio de completitud (después): ~93 % (incluye el chat en tiempo real de la entrega actual)

Módulo faltante 1: Centro de Notificaciones

Problema identificado: El backend ya tenía un endpoint funcional (`GET /api/users/{userId}/notifications`) con su modelo Pydantic (`NotificationModel`) y su servicio (`notification_service.py`), pero la aplicación móvil no tenía ninguna pantalla ni interfaz para consultar o visualizar notificaciones. El usuario no tenía forma de ver alertas de solicitudes de amistad, recordatorios de riego, o logros desbloqueados.

Impacto: - Sin acceso a notificaciones, el sistema de amigos pierde utilidad: el usuario no sabe cuándo alguien le envió una solicitud sin navegar manualmente a “Amigos”. - Las notificaciones de riego (futuras) requieren una pantalla receptora. - La UI estaba incompleta respecto al modelo de datos ya definido.

Solución implementada: Se creó el módulo completo de notificaciones en

`apps/mobile/src/features/notifications/ :`

```
features/notifications/
├─ types/notification.types.ts      - Interfaz AppNotification (TypeScript)
├─ services/notification.service.ts - getByUser() contra el API
└─ screens/NotificationsScreen.tsx  - Pantalla completa con lista de notificaciones
```

La pantalla incluye: - Lista de notificaciones con ícono contextual por tipo (solicitud de amistad, riego, logro, detección de planta) - Indicador visual de notificaciones no leídas (punto de color) - Contador de "sin leer" en un banner superior - Pull-to-refresh para actualizar - Estado vacío con instrucciones - Acceso desde el ícono de campana (🔔) en el header del Dashboard, con badge numérico

Módulo faltante 2: Acceso a notificaciones desde el Dashboard

Problema identificado: No había ningún punto de entrada visible a las notificaciones. El header del Dashboard no tenía acciones y el usuario debía navegar manualmente.

Solución implementada: Se agregó un botón de campana (🔔) en el `rightSlot` del `ScreenHeader` del Dashboard, con un badge numérico rojo que muestra las notificaciones sin leer. Este badge desaparece cuando todas están leídas.

1.2 Funcionalidad propia: Modo Vacaciones

¿Cuál es la funcionalidad?

“**Modo Vacaciones**” es una funcionalidad exclusiva de MyPlantCenter que ayuda al usuario a planificar el cuidado de sus plantas cuando va a estar fuera de casa por varios días.

El usuario selecciona su **fecha de salida** y su **fecha de regreso**. El sistema analiza automáticamente cada planta en su colección y determina:

1. Nivel de riesgo de cada planta durante la ausencia:

-  **Alto riesgo:** La planta necesitará agua más de una vez durante el viaje y quedará varios días sin ser regada.
-  **Riesgo medio:** La planta necesitará agua justo durante el período de ausencia.
-  **Sin riesgo:** La planta no necesita riego durante las vacaciones.

2. **Acción recomendada** para cada planta (ej: "Riega antes de salir — quedará sin agua varios días").
3. Una **lista de consejos prácticos** (agrupar plantas, usar platillos con agua, pedir ayuda a alguien).
4. Opción de **activar el modo vacaciones**, que persiste en el dispositivo (AsyncStorage) y muestra un **banner activo** en el Dashboard mientras las vacaciones están vigentes.

¿Qué valor aporta al usuario final?

La mayoría de las personas que tienen plantas en casa se enfrentan al mismo problema cada vez que salen de viaje: ¿cuáles plantas necesitan agua antes de irse? ¿Cuáles pueden esperar? Esta incertidumbre causa que las personas olviden regar algunas plantas críticas o rieguen en exceso otras.

MyPlantCenter ya registra la **frecuencia de riego** y la **última vez que se regó** cada planta. El Modo Vacaciones aprovecha exactamente esos datos para hacer un cálculo inteligente:

"Tu **Pothos** debe regarse cada 5 días y lleva 3 días sin agua. Si sales por 7 días, le faltarán 5 días de riego → **alto riesgo**."

Diferenciadores clave respecto a otras apps de plantas:

1. **Contextual y personalizado:** No es un recordatorio genérico — el cálculo es específico a los datos reales de cada planta del usuario.
2. **Accionable:** El resultado no es solo información, es una lista de acciones concretas a tomar antes de salir.
3. **Persistente:** El modo queda activo en el dispositivo y el Dashboard muestra un indicador visible mientras el usuario está de viaje.
4. **Sin backend adicional:** La funcionalidad es completamente local/calculada en el cliente, usando datos ya existentes. Esto la hace rápida, privada y funciona offline.

Arquitectura de la funcionalidad

[DashboardV2]
↓ navega a


```

[VacationModeScreen]
  ↓ carga plantas desde
[plant.service.ts] → GET /api/users/{userId}/plants
  ↓ persiste plan en
[vacation.service.ts] → AsyncStorage ('vacation_plan_v1')
  ↓ calcula riesgos con
[vacationService.assessRisks()]
  ↓ muestra resultado en
[VacationModeScreen → RiskRow components]
  ↓ cuando activo, muestra banner en
[DashboardV2]

```

Fórmula de cálculo de riesgo:

```

dueDate = lastWatered + wateringFrequencyDays
daysUntilDue = dueDate - today
vacationDays = returnDate - departureDate
willNeedWatering = daysUntilDue <= vacationDays
daysOverdue = vacationDays - daysUntilDue

risk = daysOverdue > 3 ? 'high' :
      daysOverdue > 0 ? 'medium' : 'low'

```

1.3 Evaluación de completitud

Estado al momento de la entrega (27/05/2026):

Área	Completitud estimada	Justificación
Flujos funcionales core	95 %	Autenticación, plantas, cámara, riego, amigos, explorar — todos operativos
UI / Diseño	98 %	Sistema de diseño completo, dark mode, navegación fluida
Notificaciones	90 %	Pantalla implementada; push notifications (FCM) pendiente

Área	Compleitud estimada	Justificación
Funcionalidad propia	100 %	Modo Vacaciones completamente funcional
Total estimado	~92 %	Por encima del umbral de 90 % solicitado

Funcionalidades pendientes para el 100 %: - Push notifications (FCM / Expo Notifications) para notificaciones en tiempo real - Subida de avatar a Firebase Storage (actualmente solo local) - Scroll infinito en la pantalla de Explorar - Búsqueda fuzzy por nombre de usuario en amigos

Parte 2: Desarrollo

2.1 Módulos implementados en esta entrega

Centro de Notificaciones

Archivos nuevos:

Archivo	Descripción
<code>features/notifications/types/notification.types.ts</code>	Interfaz <code>AppNotification</code> mapeada al <code>NotificationModel</code> del backend
<code>features/notifications/services/notification.service.ts</code>	<code>getByUser(userId)</code> — consulta <code>GET /api/users/{userId}/notifications</code>
<code>features/notifications/screens/NotificationsScreen.tsx</code>	Pantalla completa de notificaciones con lista, badges e íconos contextuales

Archivos modificados:

Archivo	Cambio
<code>core/navigation/AppNavigator.tsx</code>	Se añadió <code>Notifications: undefined</code> al <code>RootStackParamList</code> y se registró la pantalla en el stack
<code>features/dashboard/screens/Dashboard/DashboardV2.tsx</code>	Se añadió campana con badge en el header; carga del conteo de notificaciones sin leer en <code>fetchData()</code>

Decisiones técnicas: - Se normalizó la respuesta del endpoint con un guard de tipo en el servicio: el backend puede devolver el array directamente o envuelto en un `ApiResponse<T>`, por lo que el servicio maneja ambos casos sin romper. - Los íconos de notificación se mapean por `type` (string) con un diccionario de fallback a `'bell'`, lo que hace la pantalla resiliente ante tipos de notificación futuros. - El badge del Dashboard solo se muestra cuando `unreadCount > 0` para no añadir ruido visual innecesario.

Modo Vacaciones (funcionalidad propia)

Archivos nuevos:

Archivo	Descripción
<code>features/vacation/types/vacation.types.ts</code>	Interfaces <code>PlantVacationRisk</code> , <code>VacationPlan</code> , <code>PlantRiskLevel</code>
<code>features/vacation/services/vacation.service.ts</code>	Persistencia en AsyncStorage + algoritmo <code>assessRisks()</code>
<code>features/vacation/screens/VacationModeScreen.tsx</code>	Pantalla completa: selector de fechas, evaluación de riesgos, consejos

Archivos modificados:

Archivo	Cambio
<code>core/navigation/AppNavigator.tsx</code>	Se añadió <code>VacationMode: undefined</code> al stack y se registró la pantalla

Archivo	Cambio
<code>features/dashboard/screens/Dashboard/DashboardV2.tsx</code>	Se añadió card "Modo Vacaciones" con estado activo/inactivo y botón de navegación

Decisiones técnicas:

- `vacationService.assessRisks()` es una función pura (sin efectos secundarios): recibe el array de plantas y las fechas, devuelve los riesgos ordenados. Esto facilita pruebas unitarias futuras.
- La persistencia usa `AsyncStorage` con la clave `'vacation_plan_v1'` (versionada) para facilitar migraciones del esquema sin romper datos existentes.
- El selector de fechas usa `@react-native-community/datetimepicker` (ya instalado en el proyecto para `EditPlantV2`), por lo que no se añadió ninguna dependencia nueva.
- La pantalla mantiene el estado de activación sincronizado con el Dashboard a través de `useFocusEffect`, que recarga el estado al volver de la pantalla de vacaciones.

Parte 3: Entrega

Repositorio GitHub: <https://github.com/EstebanJavierGranadosSibaja/MyPlantCenter>

Branch de entrega: `develop`

Backends en línea (activos durante la revisión)

Servicio	URL pública	Estado
API de plantas (FastAPI)	<code>https://myplantcenter.onrender.com</code>	En línea
Backend de chat (FastAPI + WebSocket)	<code>https://chat-backend-4nzg.onrender.com</code>	En línea

Panel de Render (API de plantas): <https://dashboard.render.com/web/srv-d898bcegvqtc73bo33jg>

Ambos backends están desplegados en **Render**. En el plan gratuito el servicio "duerme" tras un periodo de inactividad, por lo que **la primera petición puede**

tardar ~30–60 s en responder mientras despierta. Conviene abrir ambas URLs unos minutos antes de la revisión para tenerlos “calientes”.

Build de preview para Android

Enlace de instalación del APK (Expo / EAS):

<https://expo.dev/accounts/estebanjgs/projects/myplantcenter/builds/05a8d9a7-145a-44c6-ad89-b994699a1452>

Abre ese enlace desde un dispositivo Android (o escanea el código QR de la página) para descargar e instalar la aplicación. El build se generó con:

```
cd apps/mobile
eas build -p android --profile preview
```

- **Perfil:** `preview` (distribución interna, APK instalable directamente).
- **Build ID:** `05a8d9a7-145a-44c6-ad89-b994699a1452`

Resumen de archivos nuevos del Laboratorio 4 (Notificaciones + Modo Vacaciones)

Archivo	Tipo
<code>apps/mobile/src/features/notifications/types/notification.types.ts</code>	Nuevo
<code>apps/mobile/src/features/notifications/services/notification.service.ts</code>	Nuevo
<code>apps/mobile/src/features/notifications/screens/NotificationsScreen.tsx</code>	Nuevo
<code>apps/mobile/src/features/vacation/types/vacation.types.ts</code>	Nuevo
<code>apps/mobile/src/features/vacation/services/vacation.service.ts</code>	Nuevo
<code>apps/mobile/src/features/vacation/screens/VacationModeScreen.tsx</code>	Nuevo
<code>apps/mobile/src/core/navigation/AppNavigator.tsx</code>	Modificado

Archivo	Tipo
apps/mobile/src/features/dashboard/screens/Dashboard/DashboardV2.tsx	Modificado